perplexity

## Structure of All Types of QUIC Packets

QUIC is a modern, UDP-based transport protocol designed for secure, multiplexed connections. Its packet structure is highly flexible, supporting multiple packet types with different header formats and cryptographic contexts. Below is a detailed breakdown of all QUIC packet types, their structures, and their roles in the protocol lifecycle.

### 1. QUIC Packet Header Types

QUIC defines two main header formats:

| Header Type | Usage Phase | Packet Types Included |
|---|---|---|
| **Long Header** | Connection establishment | Initial, 0-RTT, Handshake, Retry, Version Negotiation |
| **Short Header** | Data transfer (1-RTT) | 1-RTT (application data) |

The first bit of the QUIC packet indicates the header form (long or short) [1] [2] [3].

### 2. Long Header Packet Types and Structures

Long headers are used during connection setup and early data transfer. Each long header packet starts with:

- Header Form (1 bit, always 1 for long header)
- Fixed Bit (1 bit)
- Long Packet Type (2 bits)
- Reserved Bits (2 bits)
- Packet Number Length (2 bits)
- Version (32 bits)
- Destination Connection ID Length & Value
- Source Connection ID Length & Value

#### 2.1 Initial Packet

- **Purpose:** Client/server handshake initiation; carries the TLS ClientHello.
- **Structure:**

```
+-------------------------------------------------------+
| UDP Header                                            |
+-------------------------------------------------------+
```

```
| QUIC Long Header (Type=Initial, Version, DCID, SCID)     |
+----------------------------------------------------------+
| Token Length, Token (optional)                           |
+----------------------------------------------------------+
| Length                                                   |
+----------------------------------------------------------+
| Packet Number                                            |
+----------------------------------------------------------+
| Encrypted Payload:                                       |
|   - CRYPTO Frame (TLS ClientHello/ServerHello)           |
|   - PADDING Frame                                        |
+----------------------------------------------------------+
```

- **Notes:** The payload is encrypted with initial secrets derived from the DCID and version; header fields are mostly visible[1] [3] [4].

### 2.2 0-RTT Packet

- **Purpose:** Allows clients to send data with previous session keys before handshake completes.
- **Structure:** Similar to Initial, but payload contains 0-RTT data frames.

### 2.3 Handshake Packet

- **Purpose:** Carries handshake messages after Initial (e.g., TLS Finished).
- **Structure:** Like Initial, but with handshake-level encryption and different packet type bits.

### 2.4 Retry Packet

- **Purpose:** Sent by the server to request the client to prove address ownership.
- **Structure:**

```
+----------------------------------------------------------+
| QUIC Long Header (Type=Retry, Version, DCID, SCID)       |
+----------------------------------------------------------+
| Retry Token                                              |
+----------------------------------------------------------+
| Retry Integrity Tag                                      |
+----------------------------------------------------------+
```

- **Notes:** Not encrypted, but integrity-protected[1] [2].

### 2.5 Version Negotiation Packet

- **Purpose:** Server tells client which QUIC versions it supports.
- **Structure:**

```
+----------------------------------------------------------+
| QUIC Long Header (Type=Version Negotiation, DCID, SCID)  |
+----------------------------------------------------------+
```

```
| Supported Versions List                                  |
+----------------------------------------------------------+
```

- **Notes:** Not encrypted; used only at the start of connection[1] [2].

### 3. Short Header Packet (1-RTT Data)

Used after handshake for application data transfer.

- **Header Fields:**
  - Header Form (0)
  - Key Phase (1 bit)
  - Spin Bit (optional, 1 bit)
  - Reserved Bits (2 bits)
  - Packet Number Length (2 bits)
  - Destination Connection ID
  - Packet Number
  - Encrypted Payload (application data frames)
- **Notes:** Minimal header for efficiency; all payload and most header fields are encrypted[1] [2] [3].

### 4. Packet Number Spaces

QUIC uses three distinct packet number spaces:

- **Initial:** For Initial packets.
- **Handshake:** For Handshake packets.
- **Application Data:** For 0-RTT and 1-RTT packets.

Each space has its own packet number sequence[2].

### 5. Frame Types and Formats

QUIC packets carry one or more frames in their payloads. Common frame types include:

- **CRYPTO:** Carries handshake data (TLS messages).
- **STREAM:** Carries application data.
- **ACK:** Acknowledges received packets.
- **PADDING:** Used to increase packet size.
- **CONNECTION_CLOSE, MAX_DATA, MAX_STREAMS, etc.:** Control and flow management frames[1] [2] [3].

## 6. Visual Summary Table

| Packet Type | Header Type | Encrypted? | Purpose/Phase | Key Fields/Notes |
|---|---|---|---|---|
| Initial | Long | Yes | Handshake start | DCID, SCID, Version, Token, CRYPTO |
| 0-RTT | Long | Yes | Early data | DCID, SCID, Version, STREAM |
| Handshake | Long | Yes | Handshake continuation | DCID, SCID, Version, CRYPTO |
| Retry | Long | No (integrity only) | Address validation | DCID, SCID, Version, Retry Token |
| Version Negotiation | Long | No | Version negotiation | DCID, SCID, Supported Versions |
| 1-RTT | Short | Yes | Application data transfer | DCID, Packet Number, STREAM/ACK/etc. |

## 7. Lifecycle of a QUIC Connection

1. **Client → Server:** Initial Packet (CRYPTO frame with ClientHello)
2. **Server → Client:** Initial + Handshake Packets (CRYPTO frame with ServerHello, Finished)
3. **Optional:** 0-RTT data from client if resuming session
4. **Both Sides:** Switch to Short Header (1-RTT) packets for encrypted application data
5. **Connection Close:** CONNECTION_CLOSE frame

## 8. Security and Extensibility

- Most QUIC packet content is encrypted after the handshake, protecting metadata and application data from network observers.
- Only Retry and Version Negotiation packets are never encrypted.
- The protocol is versioned and extensible, with clear invariants for header fields to allow future evolution[1] [2] [3].

**References:**


- RFC 9312: Manageability of the QUIC Transport Protocol[1]
- IETF QUIC v1 Design[2]
- Colin Perkins, QUIC Transport Protocol[3]
- Parsing the QUIC Packet Description Language[4]


This structure covers all major QUIC packet types, their headers, roles, and how they fit into the protocol's lifecycle.

⁂


1. https://www.rfc-editor.org/rfc/rfc9312.pdf
2. https://www.cse.wustl.edu/~jain/cse570-21/ftp/quic/index.html
3. https://csperkins.org/teaching/2023-2024/networked-systems/lecture04/lecture04c.pdf
4. https://static1.squarespace.com/static/65d69223e3211d00da44a4cc/t/6618163e0001950152679993/1712854592523/Parsing_The_QUIC_Packet_Description_Language__Murphy_2022.pdf